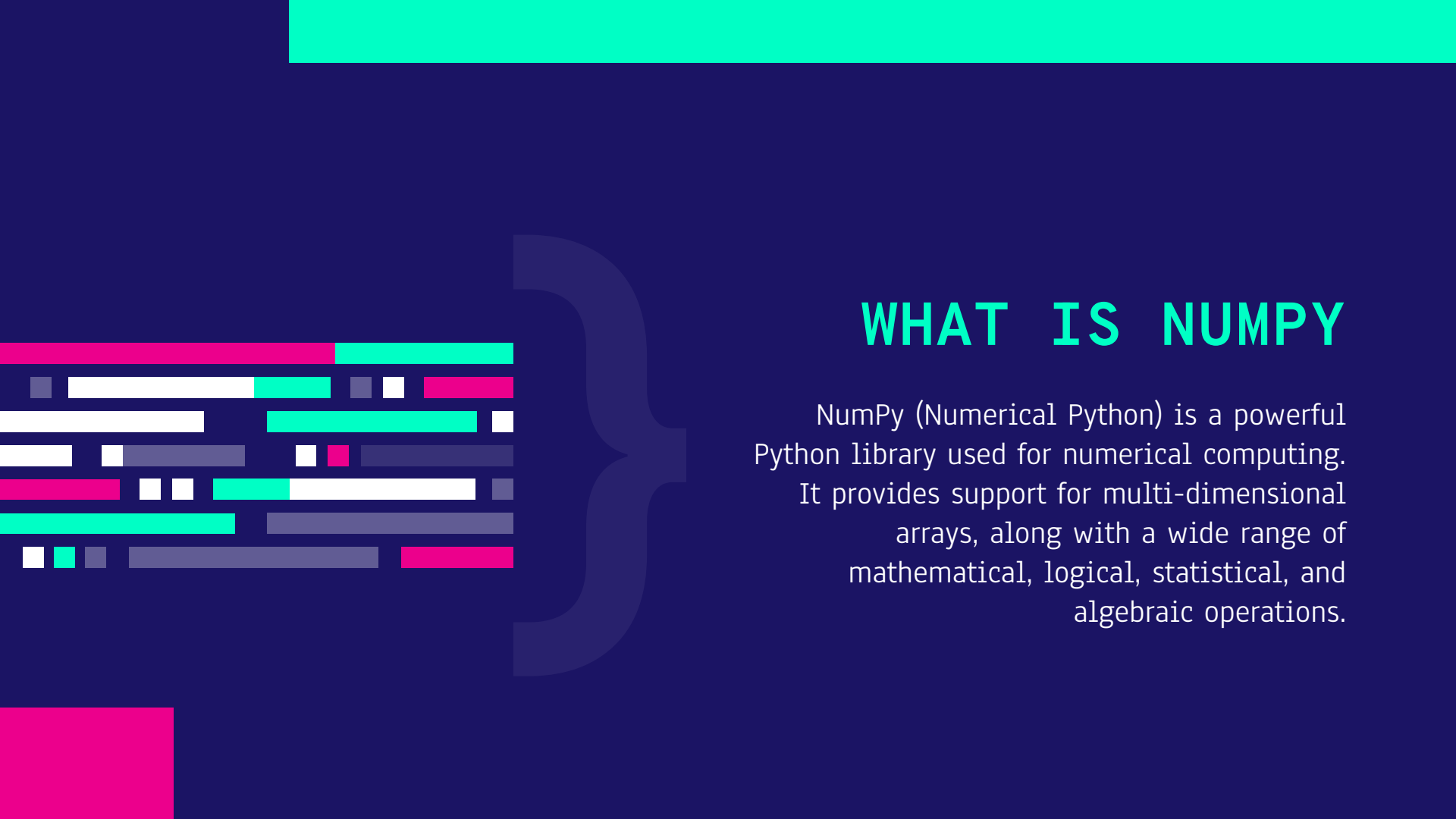




NUMPY



WHAT IS NUMPY

NumPy (Numerical Python) is a powerful Python library used for numerical computing. It provides support for multi-dimensional arrays, along with a wide range of mathematical, logical, statistical, and algebraic operations.

FEATURES OF NUMPY



Memory Efficient

Arrays use less memory compared to Python lists.

Broadcasting

Performs operations on arrays of different shapes seamlessly.



Rich Functionality

Includes built-in functions for linear algebra, stats, FFT, etc.

Fast Operations

NumPy performs operations faster using C-level optimizations.



NumPy

Common import convention:



```
1 import numpy as np
```

Creating a 1-dimensional array:

```
a1d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

NumPy



Creating a 2-dimensional array:

```
a2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

output:

```
array([[1, 2, 3],  
       [4, 5, 6],  
       [7, 8, 9]])
```

ARRAY ATTRIBUTES

ARRAY

We can query these
attributes

Shape

`array_name.shape`

Dimension

`array_name.ndim`

No of elements

`array_name.size`

Size of element

`array_name.itemsize`

Creating arrays



Creating a 2x6 array filled with 0:

```
np.zeros((2, 6))  
# or  
np.full((2, 6), 0.0)
```

Creating arrays



Creating a 2x2 array of random values:

```
# floats between 0 and 1:  
np.random.rand(2, 2)  
# integers between 1 and 6:  
np.random.randint(1, 7, size = (2, 2))
```


Array types

Each array has a predefined data type for all entries



```
a = np.array([1])  
a.dtype # int32  
b = np.array([1.0])  
b.dtype # float64  
c = np.array(['abc'])  
c.dtype # <U3  
d = np.array([b'abc'])  
d.dtype # |S3
```

COMMON ARRAY TYPES

Bool

bool / bool_ (stored as 8 bits)



Int

int8, int16, int32, int64



Float

float16, float32, float64



Selecting array entries

With 2D arrays: [row index, column index]

In general:

last index: counts rightwards

second to last index (if it exists): counts downwards

```
a1d[0] # 0
```

```
a2d[0, 1] # 2
```

```
a2d[0, :] # [1, 2, 3]
```

```
a2d[:, 0] # [1, 4, 7]
```

Slices



```
a1d[:3] # [0, 1, 2]
a1d[3:6] # [3, 4, 5]
a1d[6:] # [6, 7, 8, 9]
a1d[0:8:2] # [0, 2, 4, 6]

a1d[3:0:-1] # [3, 2, 1]
a1d[::-1] # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

a2d[1:, :] # [[5, 6, 7], [8, 9, 10]]
```

Views

Comparison: creating a copy of a list, creating a view of an array

```
list = [1, 2, 3]
list_copy = list[:]
list_copy[0] = 10 # does NOT change list
```

```
array = np.array([1, 2, 3])
array_view = array[:]
array_view[0] = 10 # DOES change array
```

Copying Arrays



Arrays can be copied via `np.copy()`

Operators

Operators are applied element-wise:

```
a = np.array([0, 1, 2, 3])  
b = np.array([2, 2, 2, 2])
```

```
-a  
# np.array([0, -1, -2, -3])  
a + b  
# np.array([2, 3, 4, 5])  
a * b  
# np.array([0, 2, 4, 6])
```

Operators

Element-wise comparison of arrays:

```
a < b
# np.array([True, True, False, False])
a == b
# np.array([False, False, True, False])
```

Warning: `a == b` cannot be used reasonably in if statements - use `np.array_equal(a, b)`

Operators

Operations with single numbers (broadcasting):

```
print(a + 1)
# np.array([1, 2, 3, 4])
```

Some constants are available directly in NumPy:

```
print(a + np.pi)
print(a + np.e)
print(np.nan)
```

Element-wise functions

NumPy provides some mathematical functions that are applied element-wise:

```
print(np.sin(a))  
# [0.0, 0.84147098, 0.9... ]  
print(np.sqrt(a))  
# [0.0, 1.0, 1.414... ]
```

Aggregation functions

Aggregations compute scalar values for an entire array or for each of its rows / columns

sum over all entries:

```
np.sum(a2d)
```

sum along axis 0 ("downwards"):

```
np.sum(a2d, axis=0)
```

sum along axis 1 ("rightwards"):

```
np.sum(a2d, axis=1)
```

EXERCISE

Given an array of prices and an array of quantities, determine the total price.

```
prices = np.array([3.99, 4.99, 3.99, 12.99])
# buying the first item 3 times and
the last item 2 times
quantities = np.array([3, 0, 0, 2])

# solution: 37.95
```

EXERCISE

Given an array of masses and velocities of some bodies, determine the kinetic energy of every body and the total kinetic energy of all bodies together.

```
masses = np.array([1.2, 2.2, 1.5,  
2.0])  
velocities = np.array([12.0, 14.0,  
14.0, 7.5])
```

EXERCISE

Given the coordinates of the vertices of a triangle, determine its centroid (arithmetic mean of its vertices)

```
a = np.array([5, 1])  
b = np.array([6, 8])  
c = np.array([1, 3])
```

```
# solution: [4, 4]
```

Vector Addition & Subtraction



```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

result = a + b
print(result) # Output: [5 7 9]
```



```
import numpy as np

a = np.array([5, 7, 9])
b = np.array([1, 2, 3])

result = a - b
print(result) # Output: [4 5 6]
```

- NumPy performs element-wise addition and subtraction
- Vectors a and b must be of the same shape

Scalar Multiplication



```
import numpy as np

a = np.array([1, 2, 3])
scalar = 3

result = scalar * a
print(result) # Output: [3 6 9]
```

- Multiplies each element by the scalar
- Common in vector scaling and transformations

Matrix Addition & Subtraction

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

result = A + B
print(result)
# Output: [[ 6  8]
#          [10 12]]
```

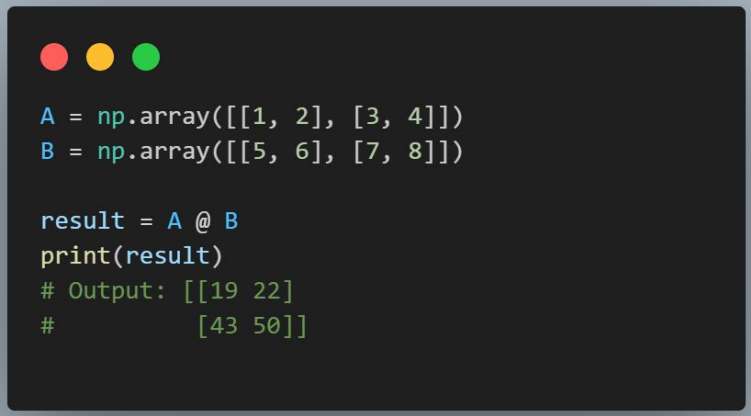
```
import numpy as np

A = np.array([[5, 6], [7, 8]])
B = np.array([[1, 2], [3, 4]])

result = A - B
print(result)
# Output: [[4 4]
#          [4 4]]
```

- Both matrices must have the same shape
- Operation is element-wise

Matrix Multiplication



```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
  
result = A @ B  
print(result)  
# Output: [[19 22]  
#          [43 50]]
```

- Use @ or np.matmul()
- Follows matrix multiplication rules (inner dimensions must match)

Matrix Inversion



```
A = np.array([[4, 7], [2, 6]])
```

```
inv_A = np.linalg.inv(A)
```

```
print(inv_A)
```

```
# Output: [[ 0.6 -0.7]
```

```
#          [-0.2  0.4]]
```

- Matrix must be square and non-singular
- Use `np.linalg.inv()` for inverse